Derivation D2: Logarithmic Height Bound for AVL Trees

Exact Derivation of $h < 1.4405 \log_2(n+2) - 0.3277$ via Minimal Fibonacci Trees

Saumya (250001064) – Role P2 (AVL Tree Engineer)
CS 253 / MA 253 Data Structures & Algorithms Lab

21 November 2026

1 Definition and Structural Invariant

An AVL tree is a binary search tree in which for every node x, the heights of its left and right subtrees differ by at most 1. Formally, let $h(x)$ denote the height of node x, defined as the number of edges on the longest path from x to a descendant leaf:

$$h(\text{nullptr}) = -1, \quad h(\text{leaf}) = 0, \quad h(x) = 1 + \max(h(x.\text{left}), h(x.\text{right})) \tag{1}$$

The balance factor $\text{bf}(x)$ is defined as:

$$\text{bf}(x) = h(x.\text{left}) - h(x.\text{right}) \tag{2}$$

The AVL balance invariant requires that for every node x in the tree:

$$\text{bf}(x) \in \{-1, 0, +1\} \tag{3}$$

2 Minimal AVL Trees and Fibonacci Recurrence

To determine the maximum height h that an AVL tree containing n nodes can attain, we analyze the dual problem: *What is the minimum number of nodes, $N(h)$, required to construct an AVL tree of height h?*

For an AVL tree of height h to contain the absolute minimum number of nodes, its root must have one child subtree of minimal height $h-1$ and another child subtree of minimal height $h-2$:

$$N(h) = 1 + N(h-1) + N(h-2) \quad \text{for } h \geq 1 \tag{4}$$

with boundary conditions:

$$N(-1) = 0 \quad \text{(empty tree)}, \qquad N(0) = 1 \quad \text{(single-node tree)} \tag{5}$$

Computing the initial terms:

$$\begin{aligned}
N(1) &= 1 + N(0) + N(-1) = 1 + 1 + 0 = 2 \\
N(2) &= 1 + N(1) + N(0) = 1 + 2 + 1 = 4 \\
N(3) &= 1 + N(2) + N(1) = 1 + 4 + 2 = 7 \\
N(4) &= 1 + N(3) + N(2) = 1 + 7 + 4 = 12 \\
N(5) &= 1 + N(4) + N(3) = 1 + 12 + 7 = 20
\end{aligned}$$

Lemma 1 *For all $h \geq -1$, $N(h) = F_{h+3} - 1$, where F_k is the k-th Fibonacci number ($F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, \ldots$).*

Proof: By mathematical induction on h:

- **Base cases:** For $h = -1$, $N(-1) = 0$ and $F_{-1+3} - 1 = F_2 - 1 = 1 - 1 = 0$. For $h = 0$, $N(0) = 1$ and $F_{0+3} - 1 = F_3 - 1 = 2 - 1 = 1$. Both hold.
- **Inductive step:** Assume $N(k) = F_{k+3} - 1$ holds for all $k < h$. Then:

$$\begin{aligned} N(h) &= 1 + N(h-1) + N(h-2) \\ &= 1 + (F_{h+2} - 1) + (F_{h+1} - 1) \\ &= F_{h+2} + F_{h+1} - 1 \\ &= F_{h+3} - 1 \end{aligned}$$

  by the standard Fibonacci identity $F_k = F_{k-1} + F_{k-2}$. Hence the identity holds for all $h \geq -1$.

■

3 Exact Closed-Form Height Bound

By Binet's formula, the k-th Fibonacci number is given by:

$$F_k = \frac{\varphi^k - \psi^k}{\sqrt{5}} \tag{6}$$

where $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887$ (Golden Ratio) and $\psi = \frac{1-\sqrt{5}}{2} \approx -0.6180339887$.

Since $|\psi| < 1$, for all $k \geq 1$ we have $|\psi^k/\sqrt{5}| \leq \frac{1}{\sqrt{5}} < 0.44721$. Thus:

$$F_{h+3} > \frac{\varphi^{h+3}}{\sqrt{5}} - 1 \tag{7}$$

Substituting into the minimal node relation for an AVL tree with n nodes:

$$n \geq N(h) = F_{h+3} - 1 > \frac{\varphi^{h+3}}{\sqrt{5}} - 2 \tag{8}$$

Rearranging gives:

$$n + 2 > \frac{\varphi^{h+3}}{\sqrt{5}} \tag{9}$$

Taking base-2 logarithms on both sides:

$$\log_2(n+2) > (h+3)\log_2(\varphi) - \frac{1}{2}\log_2(5) \tag{10}$$

$$h + 3 < \frac{\log_2(n+2) + \frac{1}{2}\log_2(5)}{\log_2(\varphi)} \tag{11}$$

$$h < \frac{1}{\log_2(\varphi)}\log_2(n+2) + \left[\frac{\log_2(5)}{2\log_2(\varphi)} - 3\right] \tag{12}$$

Numerical Constant Evaluation:
$\varphi \approx 1.6180339887 \implies \log_2(\varphi) \approx 0.6942419136$
$\frac{1}{\log_2(\varphi)} \approx 1.4404200904$
$\frac{\log_2(5)}{2\log_2(\varphi)} - 3 = \frac{2.3219280949}{2\times 0.6942419136} - 3 = 1.672275 - 3 = -1.327725$
Rounding upward for a strictly conservative ceiling bound gives:

$$h < 1.4405 \log_2(n+2) - 0.3277 \tag{13}$$

4 Implications for Operation Bounds

For any AVL tree storing n events:

- For $n = 1{,}000$, $h < 1.4405 \times 9.9687 - 0.3277 \approx 14.03 \implies h \leq 14$.
- For $n = 1{,}000{,}000$, $h < 1.4405 \times 19.9316 - 0.3277 \approx 28.38 \implies h \leq 28$.

All operations (insert, deleteMin, cancel, and successor(t)) traverse at most $h + 1 \leq 29$ pointers, strictly guaranteeing $O(\log n)$ worst-case time with no performance degradation.